基于Spark的音乐专辑数据分析展示

https://dblab.xmu.edu.cn/blog/2738/

本实验采用Python语言，使用大数据处理框架Spark对音乐专辑数据进行处理分析，并对分析结果进行可视化。

一、实验环境

（1）Linux： Ubuntu 16.04
（2）Python: 3.6
（3）Hadoop：3.1.3（查看安装教程）
（4）Spark: 2.4.0 （查看安装教程）
（5）Web框架：flask 1.0.3
（6）可视化工具：Echarts
（7）开发工具：Visual Studio Code

安装完上述环境以后，为了支持Python可视化分析，还需要执行如下命令安装Flask组件：

```
sudo apt-get install python3-pip
pip3 install flask
```

Shell 命令

本实验涉及到的所有数据集和代码，可以从百度网盘下载（下载地址，提取码：jp0j）。

二、实验数据集

1.数据集说明

在Kaggle数据平台上下载了数据集albums.csv。其中包含了10万条音乐专辑的数据。主要字段说明如下：
1) album_title：音乐专辑名称
2) genre：专辑类型
3) year_of_pub： 专辑发行年份
4) num_of_tracks： 每张专辑中单曲数量
5) num_of_sales：专辑销量
6) rolling_stone_critic：滚石网站的评分
7) mtv_critic：全球最大音乐电视网MTV的评分
8) music_maniac_critic：音乐达人的评分

2.将数据集存放在分布式文件系统HDFS中

A. 启动Hadoop中的HDFS组件，在命令行运行下面命令：

```
/usr/local/hadoop/sbin/start-dfs.sh
```
Shell 命令

B. 在hadoop上登录用户创建目录，在命令行运行下面命令：

```
hdfs dfs -mkdir -p /user/hadoop
```
Shell 命令

C. 把本地文件系统中的数据集albums.csv上传到分布式文件系统HDFS中：

```
hdfs dfs -put albums.csv
```
Shell 命令

三、pyspark的简单分析

1.建立工程文件

A. 创建文件夹code。
B. 在code下创建main.py文件。

C. 在code下创建static文件夹，存放静态文件。
D. 在code/static文件夹下面创建data目录，存放分析生成的json数据。

2.进行数据分析

本文对音乐专辑数据集albums.csv进行了一系列的分析，包括：
1) 统计各类型专辑的数量；
2) 统计各类型专辑的销量总数；
3) 统计近20年每年发行的专辑数量和单曲数量；
4) 分析总销量前五的专辑类型的各年份销量；
5) 分析总销量前五的专辑类型，在不同评分体系中的平均评分。

3.具体代码

在main.py中复制以下代码：

```
from pyspark import SparkContext
from pyspark.sql import SparkSession
import json

#统计各类型专辑的数量（只显示总数量大于2000的十种专辑类型）
def genre(sc, spark, df):
    #按照genre字段统计每个类型的专辑总数，过滤出其中数量大于2000的记录
    #并取出10种类型用于显示
    j = df.groupBy('genre').count().filter('count > 2000').take(10)
    #把list数据转换成json字符串，并写入到static/data目录下的json文件中
    f = open('static/data/genre.json', 'w')
    f.write(json.dumps(j))
    f.close()
```

```
#统计各个类型专辑的销量总数
def genreSales(sc, spark, df):
    j = df.select('genre', 'num_of_sales').rdd\
        .map(lambda v: (v.genre, int(v.num_of_sales)))\
            .reduceByKey(lambda x, y: x + y).collect()
    f = open('static/data/genre-sales.json', 'w')
    f.write(json.dumps(j))
    f.close()

#统计每年发行的专辑数量和单曲数量
def yearTracksAndSales(sc, spark, df):
    #把相同年份的专辑数和单曲数量相加，并按照年份排序
    result = df.select('year_of_pub', 'num_of_tracks').rdd\
        .map(lambda v: (int(v.year_of_pub), [int(v.num_of_tracks), 1]))\
            .reduceByKey(lambda x, y: [x[0] + y[0], x[1] + y[1]])\
                .sortByKey()\
                .collect()

    #为了方便可视化实现，将列表中的每一个字段分别存储
    ans = {}
    ans['years'] = list(map(lambda v: v[0], result))
    ans['tracks'] = list(map(lambda v: v[1][0], result))
    ans['albums'] = list(map(lambda v: v[1][1], result))
    f = open('static/data/year-tracks-and-sales.json', 'w')
    f.write(json.dumps(ans))
```

```
    f.close()

#取出总销量排名前五的专辑类型
def GenreList(sc, spark, df):
    genre_list = df.groupBy('genre').count()\
        .orderBy('count',ascending =
False).rdd.map(lambda v: v.genre).take(5)
    return genre_list


#分析总销量前五的类型的专辑各年份销量
def GenreYearSales(sc, spark, df,
genre_list):
    #过滤出类型为总销量前五的专辑，将相同类型、相同年
份的专辑的销量相加，并进行排序。
    result = df.select('genre',
'year_of_pub', 'num_of_sales').rdd\
        .filter(lambda v: v.genre in
genre_list)\
            .map(lambda v: ((v.genre,
int(v.year_of_pub)), int(v.num_of_sales)))\
                .reduceByKey(lambda x, y: x +
y)\
                    .sortByKey().collect()

    #为了方便可视化数据提取，将数据存储为适配可视化的
格式
    result = list(map(lambda v: [v[0][0],
v[0][1], v[1]], result))
    ans = {}
    for genre in genre_list:
        ans[genre] = list(filter(lambda v:
v[0] == genre, result))
    f = open('static/data/genre-year-
sales.json', 'w')
    f.write(json.dumps(ans))
    f.close()
```

```
#总销量前五的专辑类型，在不同评分体系中的平均评分
def GenreCritic(sc, spark, df, genre_list):
    #过滤出类型为总销量前五的专辑，将同样类型的专辑的滚石评分、mtv评分，音乐达人评分分别取平均
    result = df.select('genre', 'rolling_stone_critic', 'mtv_critic', 'music_maniac_critic').rdd\
        .filter(lambda v: v.genre in genre_list)\
        .map(lambda v: (v.genre, (float(v.rolling_stone_critic), float(v.mtv_critic), float(v.music_maniac_critic), 1)))\
        .reduceByKey(lambda x, y : (x[0] + y[0], x[1] + y[1], x[2] + y[2], x[3] + y[3]))\
        .map(lambda v: (v[0], v[1][0]/v[1][3], v[1][1]/v[1][3], v[1][2]/v[1][3])).collect()

    f = open('static/data/genre-critic.json', 'w')
    f.write(json.dumps(result))
    f.close()


#代码入口
if __name__ == "__main__":
    sc = SparkContext( 'local', 'test')
    sc.setLogLevel("WARN")
    spark = SparkSession.builder.getOrCreate()
    file = "albums.csv"
    df = spark.read.csv(file, header=True)
#dataframe

    genre_list = GenreList(sc, spark, df)
```

```
    genre(sc, spark, df)
    genreSales(sc, spark, df)
    yearTracksAndSales(sc, spark, df)
    GenreYearSales(sc, spark, df, genre_list)
    GenreCritic(sc, spark, df, genre_list)
```
Python

由于代码中已有详细注释，在此不多加解释。

4.代码运行

A. 在Ubuntu终端窗口中，用 hadoop 用户登录，在命令行运行su hadoop，并输入用户密码。
B. 进入代码所在目录。
C. 为了能够读取HDFS中的albums.csv文件，在命令行运行：

```
/usr/local/hadoop/sbin/start-dfs.sh
```
Shell 命令

D. 在命令行运行：

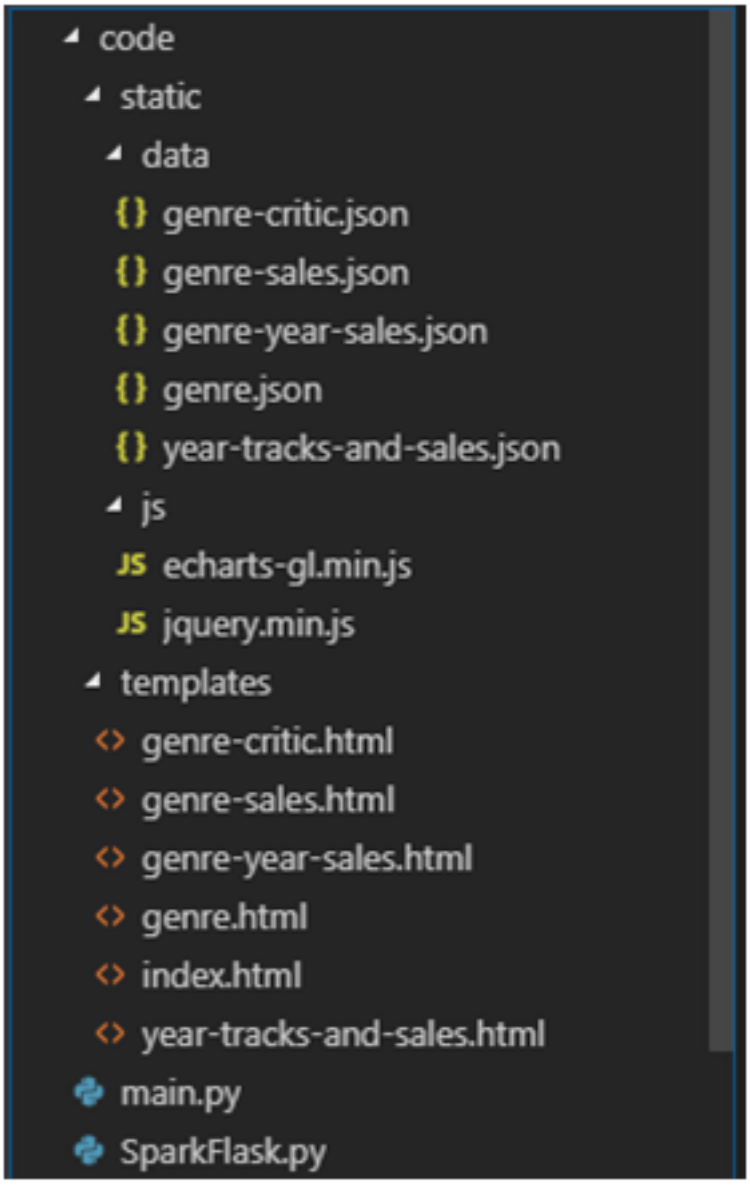
```
spark-submit main.py
```
Shell 命令

四、可视化实现

本实验的可视化基于Echarts实现，实现的可视化页面部署在基于flask框架的web服务器上。

1. 相关代码结构

1) 在code目录下新建SparkFlask.py文件，存放Flask 应用。
2) 在code目录下新建一个名为templates的文件夹，存放html文件。
3) 在code/static目录下新建一个名为js的文件夹，存放js文件。
最后的代码结构如下图所示：



2. 建立Flask应用

在SparkFlask.py文件中复制以下代码：

```
from flask import render_template
from flask import Flask
# from livereload import Server

app = Flask(__name__)

@app.route('/')
def index():
    #使用 render_template() 方法来渲染模板
```

```
    return render_template('index.html')

@app.route('/<filename>')
def req_file(filename):
    return render_template(filename)

if __name__ == '__main__':
    app.DEBUG=True#代码调试立即生效
    app.jinja_env.auto_reload = True#模板调试立即生效
    app.run()#用 run() 函数来让应用运行在本地服务器上
```
Python

代码中已有详细注释，在此不多加解释。

3.下载js文件

1) 在网站上下载jQuery（下载），将其另存为jquery.min.js文件，保存在code/static/js目录下。
2) 在官网下载界面下载Echarts（下载），将其另存echarts-gl.min.js文件，保存在code/static/js目录下。

4.Echarts画图

（1）在code/templates目录下新建index.html文件。复制以下代码：

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport"
content="width=device-width, initial-
scale=1.0">
    <meta http-equiv="X-UA-Compatible"
content="ie=edge">
```

```
    <title>Music</title>
</head>

<body>
    <h2>音乐专辑分析</h2>
    <ul style="line-height: 2em">
        <li><a href="genre.html">各类型专辑的数量统计图</a></li>
        <li><a href="genre-sales.html">各类型专辑的销量统计图</a></li>
        <li><a href="year-tracks-and-sales.html">近20年每年发行的专辑数量和单曲数量统计图</a></li>
        <li><a href="genre-year-sales.html">总销量前五的专辑类型的各年份销量分析图</a></li>
        <li><a href="genre-critic.html">总销量前五的专辑类型的评分分析图</a></li>
    </ul>
</body>
</html>
```

HTML

index.html为主页面，显示每一个统计分析图所在页面的链接。点击任意一个链接，即可跳转到相应页面。

（2）在code/templates目录下新建genre.html文件。复制以下代码：

```
<!DOCTYPE html>
<html>

<head>
    <meta charset="utf-8">
    <title>ECharts</title>
    <!-- 引入 echarts.js -->
    <script src="static/js/echarts-gl.min.js"></script>
```

```
    <script src="static/js/jquery.min.js"></script>
</head>

<body>
    <!-- 为ECharts准备一个具备大小（宽高）的Dom -->
    <a href="/">Return</a>
    <br>
    <br>
    <div id="genre" style="width: 480px;height:500px;"></div>
    <script type="text/javascript">
        $.getJSON("static/data/genre.json", d => {
            _data = d.map(v => ({
                name: v[0],
                value: v[1]
            }))
            // 基于准备好的dom，初始化echarts实例
            var myChart = echarts.init(document.getElementById('genre'), 'light');

            // 指定图表的配置项和数据
            option = {
                title: {
                    text: '各类型专辑的数量统计图',
                    subtext: '从图中可以看出Indie类型的专辑数量最多。',
                    // x: 'center'
                    x: 'left'
                },
                tooltip: {
                    trigger: 'item',
```

```
                    formatter: "{a} <br/>{b} : {c} ({d}%)"
                },
                legend: {
                    x: 'center',
                    y: 'bottom',
                    data: d.map(v => v[0])
                },
                toolbox: {
                    show: true,
                    feature: {
                        mark: { show: true },
                        dataView: { show: true, readOnly: false },
                        magicType: {
                            show: true,
                            type: ['pie', 'funnel']
                        },
                        restore: { show: true },
                        saveAsImage: { show: true }
                    }
                },
                calculable: true,
                series: [
                    {
                        name: '半径模式',
                        type: 'pie',
                        radius: [30, 180],
                        center: ['50%', '50%'],
                        roseType: 'radius',
                        label: {
                            normal: {
                                show: false
                            },
```

```
                                emphasis: {
                                    show: true
                                }
                            },
                            lableLine: {
                                normal: {
                                    show: false
                                },
                                emphasis: {
                                    show: true
                                }
                            },
                            data: _data
                        }
                    ]
                };

                // 使用刚指定的配置项和数据显示图表。
                myChart.setOption(option);
            })
    </script>
</body>
</html>
```

HTML

这个通过读取code/static/data/genre.json中的数据，画出玫瑰图，显示各类型专辑的数量。

（3）在code/templates目录下新建genre-sales.html文件。复制以下代码：

```
<!DOCTYPE html>
<html>

<head>
    <meta charset="utf-8">
    <title>ECharts</title>
    <!-- 引入 echarts.js -->
```

```
8.     <script src="static/js/echarts-gl.min.js"></script>
9.     <script src="static/js/jquery.min.js"></script>
10. </head>
11. 
12. <body>
13.     <a href="/">Return</a>
14.     <br>
15.     <br>
16.     <!-- 为ECharts准备一个具备大小（宽高）的Dom -->
17.     <div id="genre-sales" style="width: 1000px;height:550px;"></div>
18.     <script type="text/javascript">
19.         $.getJSON("static/data/genre-sales.json", d => {
20.             console.log(d);
21. 
22.             // 基于准备好的dom，初始化echarts实例
23.             var myChart = echarts.init(document.getElementById('genre-sales'), 'light');
24. 
25.             var dataAxis = d.map(v => v[0]);
26.             var data = d.map(v => parseInt(v[1])/1e6);
27. 
28.             option = {
29.                 title: {
30.                     text: '各类型专辑的销量统计图',
31.                     subtext: '该图统计了各个类型专辑的销量和，从图中可以看出 Indie 类型的专辑销量最高，将近 47 亿。Pop 类型的专辑销量排在第二，约为39亿。',
32.                     x: 'center',
```

```
                    // bottom: 10
                    padding: [0, 0, 15, 0]
                },
                color: ['#3398DB'],
                tooltip: {
                    trigger: 'axis',
                    axisPointer: {
// 坐标轴指示器，坐标轴触发有效
                        type: 'shadow'
// 默认为直线，可选为：'line' | 'shadow'
                    }
                },
                grid: {
                    left: '3%',
                    right: '4%',
                    bottom: '3%',
                    containLabel: true
                },
                xAxis: [
                    {
                        type: 'category',
                        data: dataAxis,
                        axisTick: {
                            show: true,
                            alignWithLabel:
true,
                            interval: 0
                        },
                        axisLabel: {
                            interval: 0,
                            rotate: 45,
                        }
                    }
                ],
                yAxis: [
                    {
                        type: 'value',
```

```
                        name: '# Million Albums',
                        nameLocation: 'middle',
                        nameGap: 50
                    }
                ],
                series: [
                    {
                        name: '直接访问',
                        type: 'bar',
                        barWidth: '60%',
                        data: data
                    }
                ]
            };

            // 使用刚指定的配置项和数据显示图表。
            myChart.setOption(option);

        })
    </script>
</body>
</html>
```

HTML

这个通过读取code/static/data/genre-sales.json中的数据，画出柱状图，显示各类型专辑的销量总数。

（4）在code/templates目录下新建year-tracks-and-sales.html文件。复制以下代码：

```
<!DOCTYPE html>
<html>

<head>
    <meta charset="utf-8">
    <title>ECharts</title>
```

```
    <!-- 引入 echarts.js -->
    <script src="static/js/echarts-gl.min.js"></script>
    <script src="static/js/jquery.min.js"></script>
</head>

<body>
    <a href="/">Return</a>
    <br>
    <br>
    <!-- 为ECharts准备一个具备大小（宽高）的Dom -->
    <div id="canvas" style="width: 1000px;height:550px;"></div>
    <script type="text/javascript">
        $.getJSON("static/data/year-tracks-and-sales.json", d => {
            console.log(d)
            // 基于准备好的dom，初始化echarts实例
            var myChart = echarts.init(document.getElementById('canvas'), 'light');

            var colors = ['#5793f3', '#d14a61', '#675bba'];

            option = {
                title: {
                    text: '近20年的专辑数量和单曲数量的变化趋势',
                    padding: [1, 0, 0, 15]
                    // subtext: '该图显示了从2000年到2019年发行的专辑数量和单曲数量的变化趋势，从图中可以看出，专辑数量变化很小，基本稳定在5000左右；单曲数量有轻微的波动，大概为专辑数量的10倍。'
```

```
31.                 },
32.                 tooltip: {
33.                     trigger: 'axis'
34.                 },
35.                 legend: {
36.                     data: ['单曲数量', '专辑数
   量'],
37.                     padding: [2, 0, 0, 0]
38.                 },
39.                 toolbox: {
40.                     show: true,
41.                     feature: {
42.                         dataZoom: {
43.                             yAxisIndex:
   'none'
44.                         },
45.                         dataView: { readOnly:
   false },
46.                         magicType: { type:
   ['line', 'bar'] },
47.                         restore: {},
48.                         saveAsImage: {}
49.                     }
50.                 },
51.                 xAxis: {
52.                     type: 'category',
53.                     boundaryGap: false,
54.                     data: d['years'],
55.                     boundaryGap: ['20%',
   '20%']
56.                 },
57.                 yAxis: {
58.                     type: 'value',
59.                     // type: 'log',
60.                     axisLabel: {
61.                         formatter: '{value}'
62.                     }
63.                 },
```

```
                series: [
                    {
                        name: '单曲数量',
                        type: 'bar',
                        data: d['tracks'],
                        barWidth: 15,
                    },
                    {
                        name: '专辑数量',
                        type: 'bar',
                        data: d['albums'],
                        barGap:  '-100%',
                        barWidth: 15,
                    }
                ]
            };

            // 使用刚指定的配置项和数据显示图表。
            myChart.setOption(option);

        })
    </script>
</body>
</html>
```

HTML

这个通过读取code/static/data/ year-tracks-and-sales.json中的数据，画出柱状图，显示近20年每年发行的专辑数量和单曲数量。

（5）在code/templates目录下新建genre-year-sales.html文件。复制以下代码：

```
<!DOCTYPE html>
<html>

<head>
    <meta charset="utf-8">
    <title>ECharts</title>
```

```
    <!-- 引入 echarts.js -->
    <script src="static/js/echarts-gl.min.js"></script>
    <script src="static/js/jquery.min.js"></script>
</head>

<body>
    <a href="/">Return</a>
    <br>
    <br>
    <!-- 为ECharts准备一个具备大小（宽高）的Dom -->
    <div id="genre-year-sales" style="width: 1000px;height:550px;"></div>
    <script type="text/javascript">
        $.getJSON("static/data/genre-year-sales.json", d => {
            console.log(d);

            // 基于准备好的dom，初始化echarts实例
            var myChart = echarts.init(document.getElementById('genre-year-sales'), 'light');
            option = {
                legend: {},
                tooltip: {
                    trigger: 'axis',
                    showContent: false
                },
                dataset: {
                    source: [

['year', ...d['Indie'].map(v => `${v[1]}`)],
                                ...['Indie', 'Pop', 'Rap', 'Latino', 'Pop-Rock'].map(v => [v, ...d[v].map(v1 => v1[2])])
```

```
                    ]
                },
                xAxis: { type: 'category' },
                yAxis: { gridIndex: 0 },
                grid: { top: '55%' },
                series: [
                    { type: 'line', smooth: true, seriesLayoutBy: 'row' },
                    { type: 'line', smooth: true, seriesLayoutBy: 'row' },
                    { type: 'line', smooth: true, seriesLayoutBy: 'row' },
                    { type: 'line', smooth: true, seriesLayoutBy: 'row' },
                    { type: 'line', smooth: true, seriesLayoutBy: 'row' },
                    {
                        type: 'pie',
                        id: 'pie',
                        radius: '30%',
                        center: ['50%', '25%'],
                        label: {
                            formatter: '{b}: {@2000} ({d}%)' //b是数据名，d是百分比
                        },
                        encode: {
                            itemName: 'year',
                            value: '2000',
                            tooltip: '2000'
                        }
                    }

                ]
            };

            myChart.on('updateAxisPointer', function (event) {
```

```
                var xAxisInfo = event.axesInfo[0];
                if (xAxisInfo) {
                    var dimension = xAxisInfo.value + 1;
                    myChart.setOption({
                        series: {
                            id: 'pie',
                            label: {
                                formatter: '{b}: {@[' + dimension + ']} ({d}%)'
                            },
                            encode: {
                                value: dimension,
                                tooltip: dimension
                            }
                        }
                    });
                }
            });

            // 使用刚指定的配置项和数据显示图表。
            myChart.setOption(option);

        })
    </script>
</body>
</html>
```

HTML

这个通过读取code/static/data/ genre-year-sales.json中的数据，画出扇形图和折线图，分别显示不同年份各类型专辑的销量占总销量的比例，和总销量前五的专辑类型的各年份销量变化。

（6）在code/templates目录下新建genre-critic.html文件。复制以下代码：

```
<!DOCTYPE html>
<html>

<head>
    <meta charset="utf-8">
    <title>ECharts</title>
    <!-- 引入 echarts.js -->
    <script src="static/js/echarts-gl.min.js"></script>
    <script src="static/js/jquery.min.js"></script>
</head>

<body>
    <a href="/">Return</a>
    <br>
    <br>
    <!-- 为ECharts准备一个具备大小（宽高）的Dom -->
    <div id="genre-critic" style="width: 1000px;height:550px;"></div>
    <script type="text/javascript">
        $.getJSON("static/data/genre-critic.json", d => {
            console.log(d);

            // 基于准备好的dom，初始化echarts实例
            var myChart = echarts.init(document.getElementById('genre-critic'), 'light');
            option = {
                legend: {},
                tooltip: {},
                dataset: {
                    source: [
                        ['genre', ...d.map(v => v[0])],
```

```
['rolling_stone_critic', ...d.map(v => v[1])],
['mtv_critic', ...d.map(v => v[2])],
['music_maniac_critic', ...d.map(v => v[3])]
                    ]
                },
                xAxis: [
                    { type: 'category', gridIndex: 0 },
                    { type: 'category', gridIndex: 1 }
                ],
                yAxis: [
                    { gridIndex: 0 , min: 2.7},
                    { gridIndex: 1 , min: 2.7}
                ],
                grid: [
                    { bottom: '55%' },
                    { top: '55%' }
                ],
                series: [
                    // These series are in the first grid.
                    { type: 'bar', seriesLayoutBy: 'row' , barWidth: 30},
                    { type: 'bar', seriesLayoutBy: 'row' , barWidth: 30},
                    { type: 'bar', seriesLayoutBy: 'row' , barWidth: 30 },
                    // These series are in the second grid.
                    { type: 'bar', xAxisIndex: 1, yAxisIndex: 1 , barWidth: 35},
```

```
                    { type: 'bar',
xAxisIndex: 1, yAxisIndex: 1 , barWidth: 35},
                    { type: 'bar',
xAxisIndex: 1, yAxisIndex: 1 , barWidth: 35},
                    { type: 'bar',
xAxisIndex: 1, yAxisIndex: 1 , barWidth: 35}
                ]
            };

            // 使用刚指定的配置项和数据显示图表。
            myChart.setOption(option);
        })
</script>
</body>
</html>
```
HTML

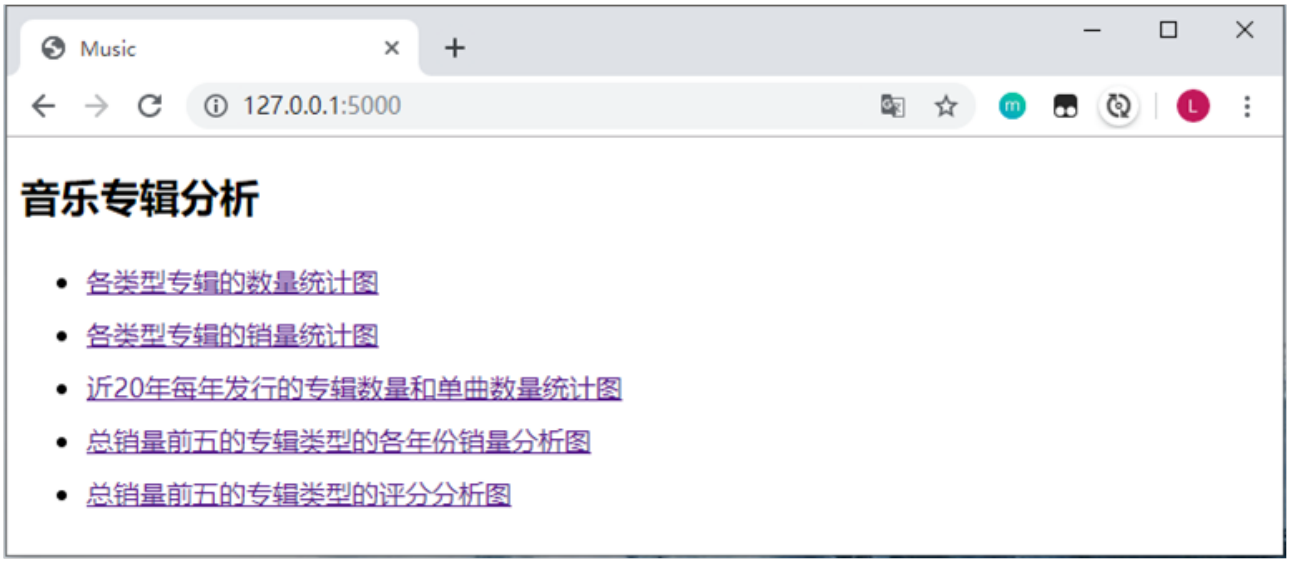
这个通过读取code/static/data/ genre-critic.json中的数据，画出柱形图，显示总销量前五的专辑类型，在不同评分体系中的平均评分。

5. 启动程序

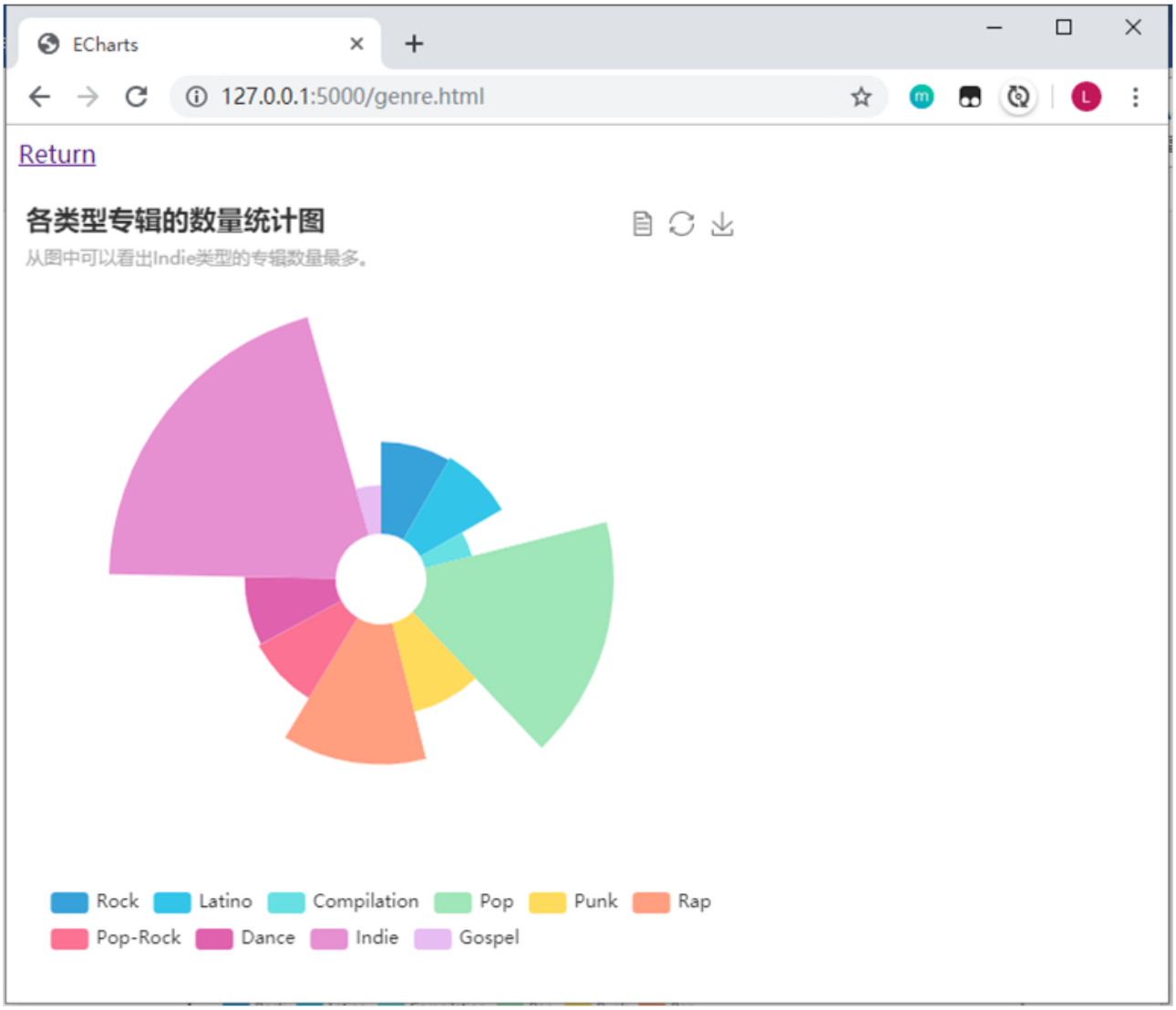
A. 在另一个Ubuntu终端窗口中，用 hadoop 用户登录，在命令行运行su hadoop，并输入用户密码。
B. 进入代码所在目录。
C. 在命令行运行如下命令：

```
spark-submit SparkFlask.py
```
Shell 命令

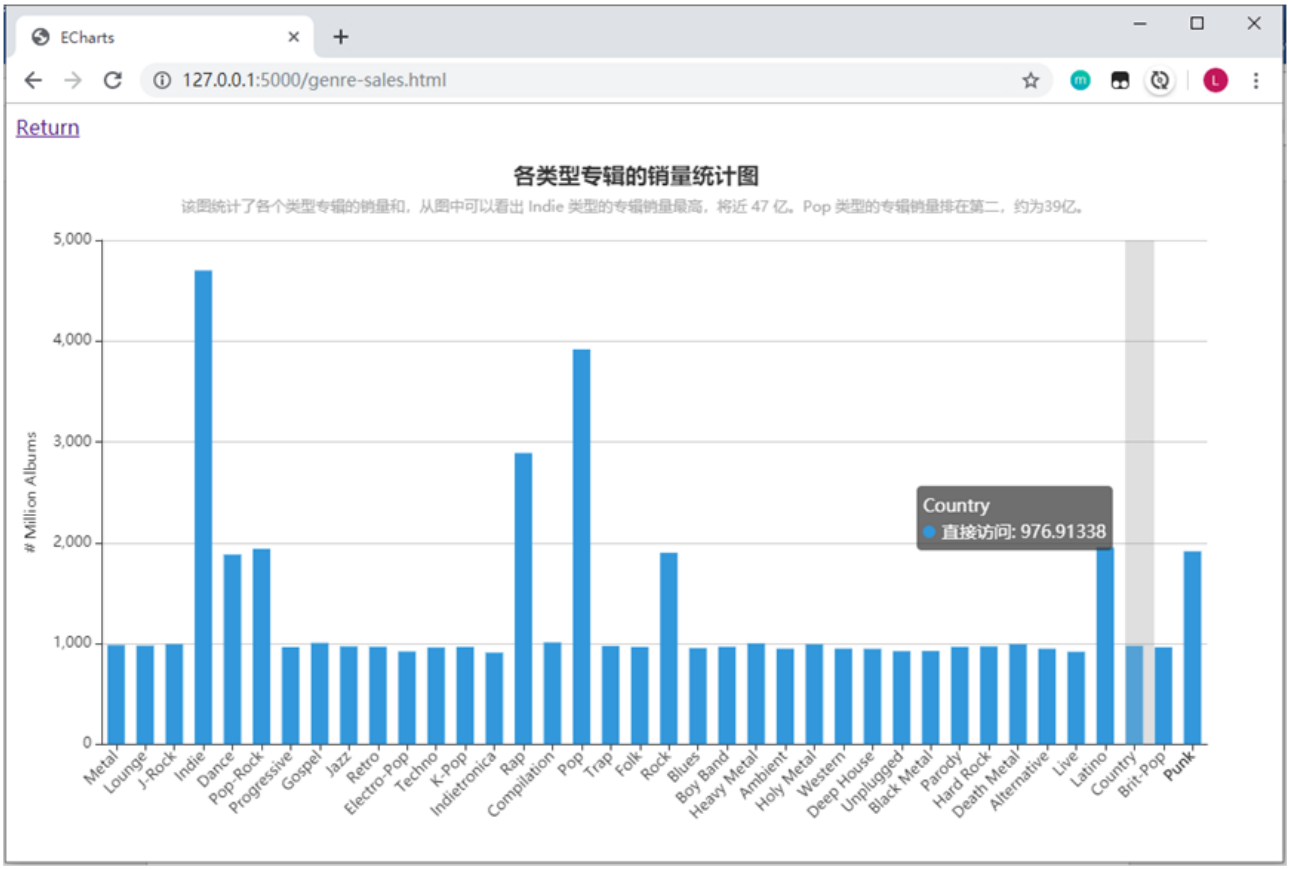
D. 在浏览器打开http://127.0.0.1:5000/，可看到如下界面：



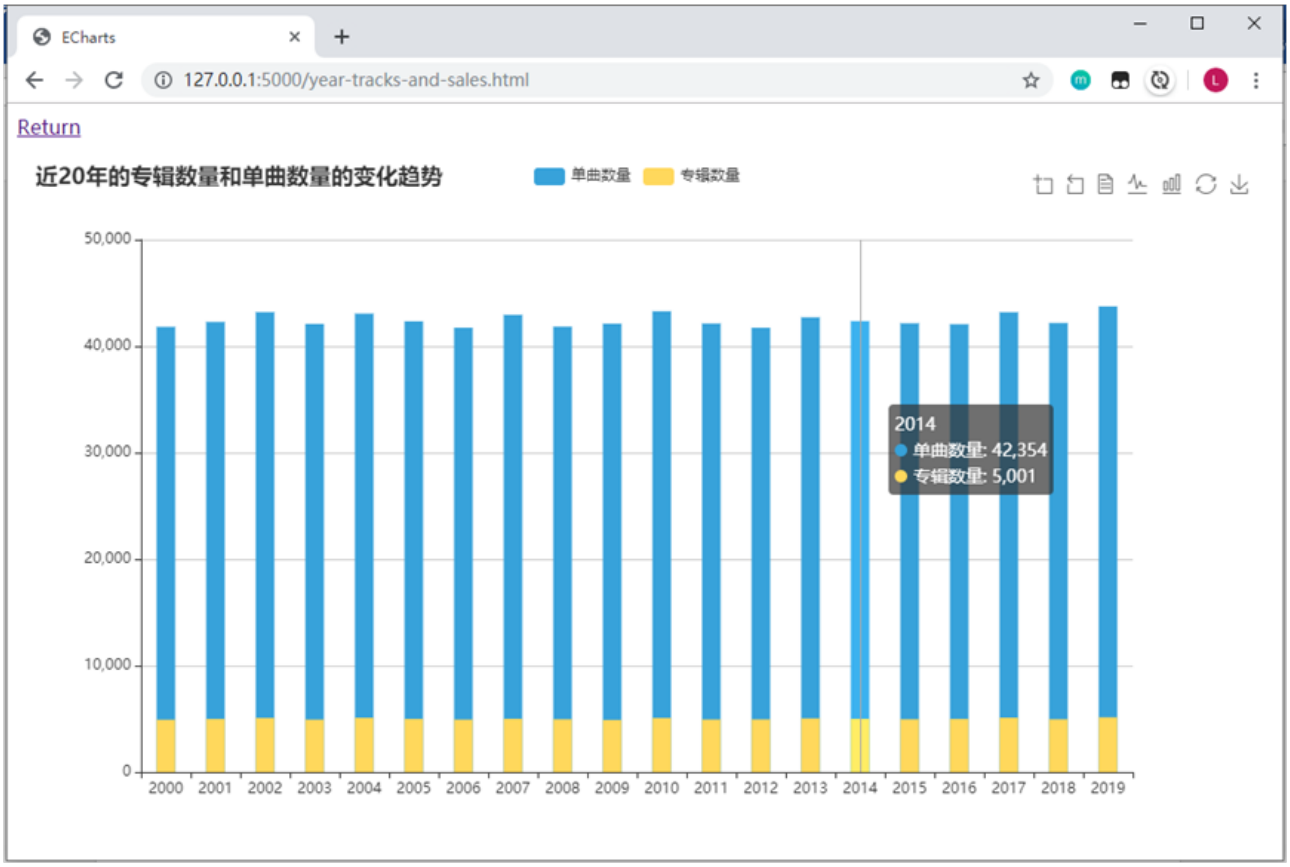
1) 各类型专辑的数量统计图



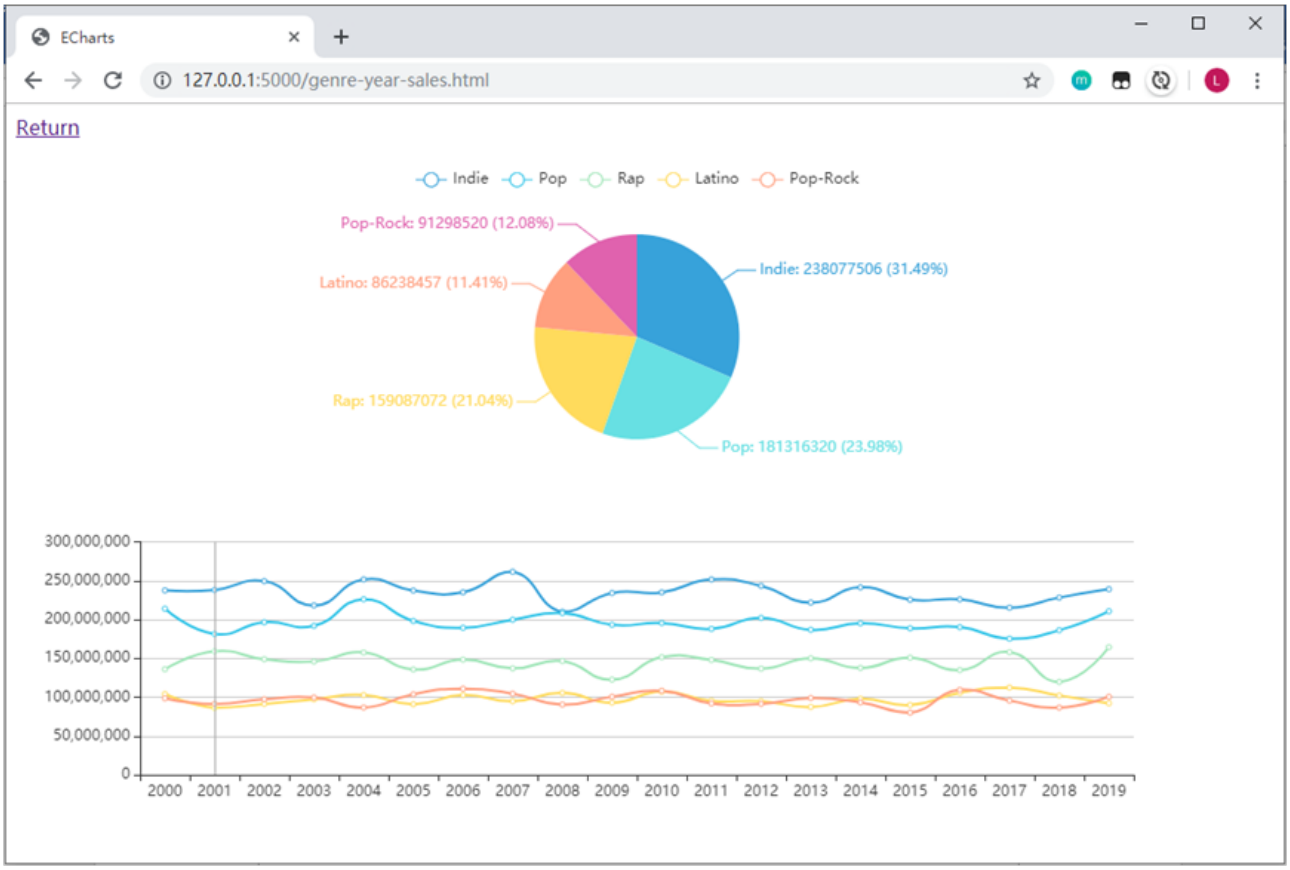
2) 各类型专辑的销量统计图



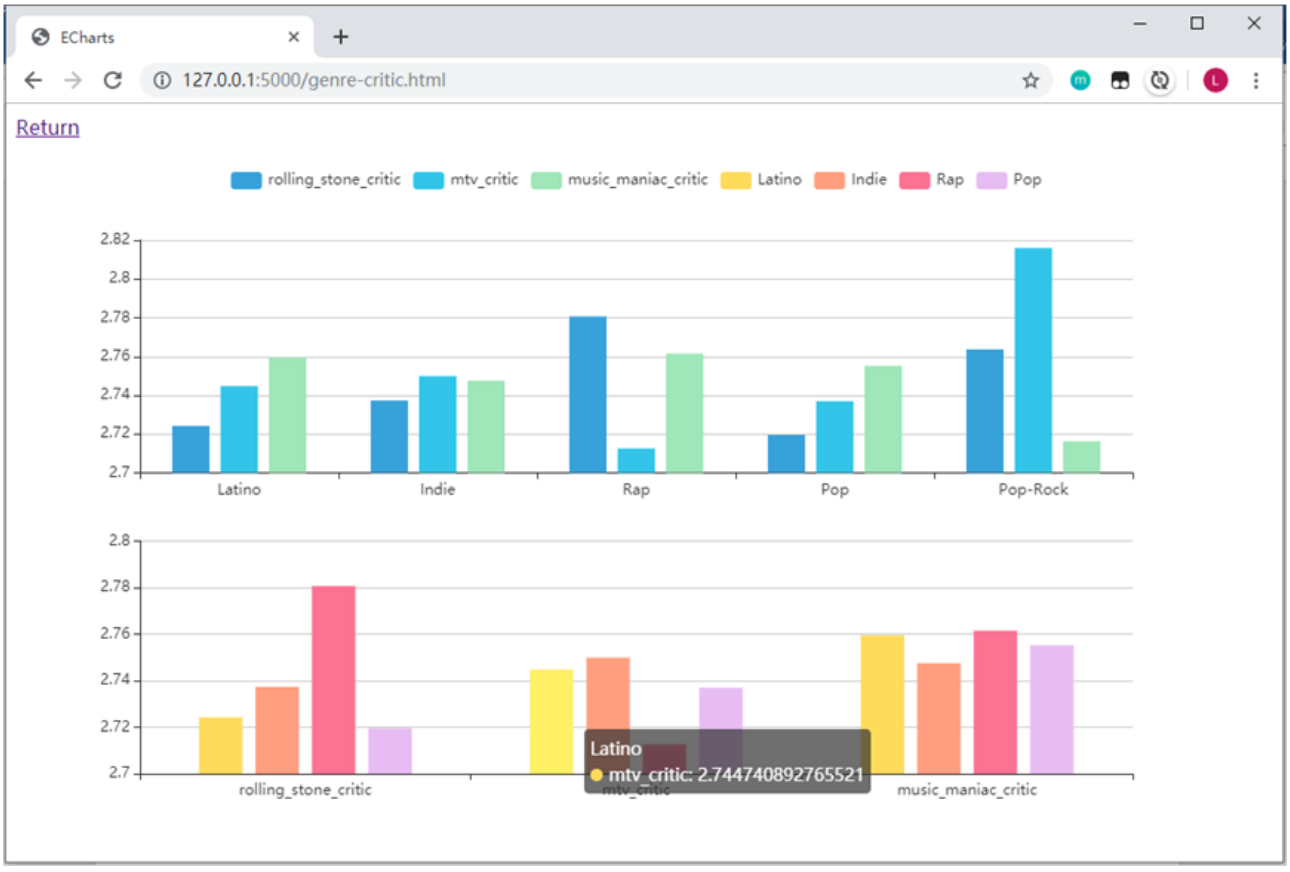
3) 近20年每年发行的专辑数量和单曲数量统计图



4) 总销量前五的专辑类型的各年份销量分析图



5) 总销量前五的专辑类型的评分分析图



本文作者